

EECS 2311 Lab 5 : Take Home Assignment

(Larissa Singh – 219308543)

Part 1 : End-to-End Manual Test Cases for User Story: Display External Resources (ITR1)

- **Test Case 1: Successful Resource Display for Valid Query**

- **Objective:** Verify that Google search results load and display correctly for a valid academic query.

- **Preconditions:** Stable internet connection.

- **Steps:**

1. Navigate to the StudyBase home page.
2. Enter "Object-Oriented Programming" in the search box.
3. Select an education level (e.g. "Year 1-2 University").
4. Click "Generate Roadmap."
5. Scroll down to the "External Resources" section.

- **Expected Results:**

- Roadmap generates successfully.
- Below the roadmap, up to 6 clickable external resource cards are displayed.
- Each card contains a favicon, hostname, and a short description that ends with a period.
- Clicking any card opens the resource in a new browser tab.

- **Test Case 2: Query with No Search Results**

- **Objective:** Verify system response when no external results are found for the given query.

- **Steps:**

1. Enter a random, gibberish string (e.g., "asdjas").
2. Click "Generate Roadmap."

- **Expected Results:**

- Roadmap shows an empty state if no topics found.
- In the "External Resources" section, a message appears: "No Google results found."

- **Test Case 3: Resource Links Functionality**

- **Objective:** Verify that each external resource opens in a new tab and does not disrupt the current session.

- **Steps:**

1. Enter "Algorithms" in the search bar.
2. Click "Generate Roadmap."
3. Once external resources appear, click each resource one by one.

- **Expected Results:**

- All links open in separate browser tabs.
- The StudyBase page remains active without any disruption.

- **Test Case 4: Handling Special Characters in Query**

- **Objective:** Ensure external resources load correctly or show a proper message when special characters are used.

- **Steps:**

1. Enter the query "#\$%^@" in the search bar.
2. Click "Generate Roadmap."

- **Expected Results:**

- Either external resources load with cleaned input, or
- "No Google results found." is displayed.

- **Test Case 5: Handling Very Long Query**

- **Objective:** Test system behavior with excessively long search queries.

- **Steps:**

1. Enter the query: "The comprehensive analysis of machine learning applications across healthcare, finance, robotics, and environmental monitoring."
2. Click "Generate Roadmap."

- **Expected Results:**

- External resources still load if results are found.
- The UI displays resource descriptions without breaking the layout.

- **Test Case 6: Network Failure Scenario**

- **Objective:** Validate error handling when network connection is unstable.

- **Steps:**

1. Throttle network to "Offline" in browser dev tools.
2. Enter "Python Programming" in the search.
3. Click "Generate Roadmap."

- **Expected Results:**

- A message: "Error fetching Google results." appears.
- No console crashes or UI freezes occur.

- **Test Case 7: Display Layout on Different Screen Sizes**

- **Objective:** Verify responsive design for resource display.

- **Steps:**

1. Perform a search on desktop.
2. Resize the window to tablet and mobile viewports.
3. Repeat on mobile devices.

- **Expected Results:**

- Resource cards realign responsively.
- No overlapping or cut-off descriptions.

- **Test Case 8: Successful YouTube Video Display**

- **Objective:** Verify that a relevant YouTube video loads for an academic query.

- **Steps:**

1. Enter "Introduction to Data Science" in the search bar.
2. Click "Generate Roadmap."
3. Scroll to the video section.

- **Expected Results:**

- A long-duration YouTube video is embedded.
- Below the video, display:

- Video title
- Channel name
- View count
- Video duration (formatted as HH:MM:SS or MM:SS)

- **Test Case 9: No Video Found Handling**

- **Objective:** Validate system behavior when no suitable video is found.

- **Steps:**

1. Enter random gibberish, e.g., "qweurioqweuri".
2. Click "Generate Roadmap."

- **Expected Results:**

- The embedded video section shows the message: "No video found."

- **Test Case 10: Playback Responsiveness**

- **Objective:** Ensure that the embedded player resizes properly across devices.

- **Steps:**

1. Search for "Operating Systems Concepts."
2. Switch between desktop, tablet, and mobile viewports.

- **Expected Results:**

- The YouTube video frame resizes and remains functional on all screen sizes.

- **Test Case 11: API Failure Simulation**

- **Objective:** Test UI behavior when YouTube API fails.

- **Steps:**

1. Disable internet connectivity.
2. Enter "Statistics" and click "Generate Roadmap."

- **Expected Results:**

- Console shows error: "Error fetching YouTube data."
- UI handles the error without freezing.

- **Test Case 12: Displaying View Counts and Durations**

- **Objective:** Verify that view counts are formatted with commas and duration is properly displayed.

- **Steps:**

1. Enter "Neural Networks Basics" and click "Generate Roadmap."
2. Observe the view count and duration display format.

- **Expected Results:**

- View counts appear like "12,345 views".
- Duration displays in readable time format.

- **Test Case 13: Handling Rapid Pagination Clicks (Race Condition Check)**

- **Objective:** Test if rapid clicks on pagination buttons cause glitches.

- **Steps:**

1. Search for "Linear Algebra."
2. Quickly click the "Next" button multiple times.
3. Observe UI responsiveness and video loading.

- **Expected Results:**

- Videos queue without UI freeze or mismatch.

Part 2: Bugs and Code Review Report

Bugs

Can Reproduce - Yes

- **Hardcoded API Keys (Security Issue) – Severity : Medium**

In both Google and YouTube code, the API keys are written directly in the JavaScript files. This is unsafe because anyone who looks at the code can take those keys and misuse them. These keys should be kept secret, either on the server or in an environment file.

- **No Retry if Something Fails – Severity : Medium**

If there is an error while getting results from Google or YouTube, the system only shows an error in the console and does not try again. There is no way for the user to retry, and they are left without results.

- **Multiple Clicks Causing Problems (Race Condition) – Severity : Low**

If the user clicks "Next" or "Previous" on the YouTube video section too many times too quickly, the system makes too many API calls at the same time. This could lead to

overlapping fetch requests and incorrect video content being displayed, causing UI glitches and state inconsistency.

- **Missing Loading Indicators – Severity : Low**
When Google or YouTube results are loading, there is no spinner or sign that something is happening. On a slow internet connection, it looks like the system is frozen.
- **Weak Error Handling for Missing Data – Severity : Low**
The YouTube API sometimes might not return all expected information, such as view counts or video duration. The code does not handle this situation, which could cause errors or missing information on the screen.
- **Inadequate Error Feedback to Users – Severity : Low**
If the API calls fail, the user does not see any clear message. The errors are only logged in the browser console, which the user cannot see.

Code Smells

- **Hardcoded Numbers and Strings**
Values such as `maxResults=1`, `videoDuration=long`, and truncation lengths (like cutting descriptions to 10 words) are hardcoded without explanation or defined constants. This makes future changes difficult and decreases code readability.
- **Long Functions**
Functions like `displayGoogleResults()` and `displayResult()` are handling multiple responsibilities, including API response parsing, HTML rendering, and event handling logic, all in one block. These large functions are difficult to maintain and test, and they violate the single-responsibility principle.
- **Duplicated Code**
The Google and YouTube sections have very similar code for fetching data and showing it. This could be combined into shared helper functions to make the code cleaner and easier to manage.
- **Global Variables**
The use of global variables such as `currentQuery`, `nextPageToken`, `prevPageToken`, and `topicCache` introduces potential risks, especially if users initiate multiple searches rapidly. This global state can lead to race conditions and unpredictable UI behavior.
- **Mixing UI and Logic**
The code for making API requests and building the HTML are combined in the same function. It's better to keep these separate so that the code is easier to read and update.
- **Not Enough Error Handling**
While console logging is present, no UI feedback is given when APIs fail or return invalid data. User-facing error alerts or fallback visuals are necessary to maintain transparency and usability.
- **Lack of modularity**
Large functions are hard to reuse. It would be better to split them into smaller pieces that can be reused in other parts of the code.

Suggested Fixes:

- Move the API keys to environment files or secure backend servers.

- Add retry buttons or automatic retry attempts when API calls fail.
- Add a spinner or loading message while waiting for results.
- Break large functions into smaller ones that only do one thing.
- Avoid repeating code by using shared helper functions.
- Use local variables instead of global ones to prevent unexpected behavior.
- Show user-friendly error messages if Google or YouTube data cannot load.
- Prevent users from clicking “Next” or “Previous” too quickly by adding button disable states or request throttling.